# U06 · Genéricos y colecciones

Ya sabes almacenar datos en arrays — pero un array tiene tamaño fijo y no sabe nada de listas ordenadas, conjuntos sin duplicados o diccionarios clave-valor. Java trae todo eso ya construido, listo para usar.

## 1. ¿Por qué genéricos?

Antes de Java 5, una estructura de datos reutilizable solo podía almacenar `Object` — y recuperar un elemento exigía un *casting* manual:

```
String item = (String) lista.get(i);   // "confía en mí, sé que ahí dentro hay un String"
```

El problema: si por error metías algo que no era un `String`, el compilador no se quejaba — el error (`ClassCastException`) aparecía **en tiempo de ejecución**, quizás mucho después y lejos de donde estaba el fallo real.

Los **genéricos** resuelven esto parametrizando el tipo: `ArrayList<String>` le dice al compilador "esta lista solo admite `String`", y el propio compilador rechaza cualquier intento de meter otra cosa — sin necesidad de casting al leer.

## 2. Clases y métodos genéricos

Puedes definir tus propias clases genéricas con un parámetro de tipo (por convenio, una letra mayúscula):

```
class Caja<T> {
    private T contenido;
    public Caja(T contenido) { this.contenido = contenido; }
    public T getContenido() { return contenido; }
}

Caja<Integer> cajaEnteros = new Caja<>(28);
Caja<String> cajaTexto = new Caja<>("Hola");
int valor = cajaEnteros.getContenido();   // sin casting
```

Letras habituales para el parámetro de tipo (por convenio, no obligatorio): `T` (tipo), `E` (elemento, típico en colecciones), `K`/`V` (clave/valor, típico en mapas), `N` (número).

### Tipos limitados (*bounded types*)

A veces necesitas que el tipo genérico tenga ciertos métodos disponibles — por ejemplo, para hacer operaciones numéricas necesitas que `T` sea, como mínimo, un `Number`:

```
class OperaMate<T extends Number> {
    private T num;
    public OperaMate(T num) { this.num = num; }
    double reciproco() { return 1.0 / num.doubleValue(); }   // ahora sí compila: T
garantiza tener doubleValue()
}
```

`<T extends Number>` significa "T puede ser `Number` o cualquier subclase suya" (`Integer`, `Double`…) — no literalmente herencia de interface, sino un límite superior sobre qué tipos son aceptables.

## 3. El framework de colecciones

Todas las estructuras de datos de Java giran en torno a la interface `Collection<T>`, con dos grandes familias:

- **`List<T>`**: secuencia ordenada, admite duplicados, accedes por posición (índice).
- **`Set<T>`**: no admite duplicados, no garantiza un orden concreto (salvo implementaciones específicas).

Y, aparte de `Collection`, la interface `Map<K,V>` para asociaciones clave-valor (no es una `Collection`, tiene su propia jerarquía).

`Collection<T>` obliga a ofrecer métodos comunes a cualquier colección — y varios de ellos tienen una lectura muy directa en términos de conjuntos matemáticos:

| Método | Qué hace | Equivalente matemático |
|---|---|---|
| `add(e)` / `remove(e)` | Añade / quita un elemento | — |
| `contains(e)` | ¿Está el elemento? | $e \in c$ |
| `addAll(c2)` | Añade todos los elementos de `c2` | Unión: $c \cup c2$ |
| `removeAll(c2)` | Quita los elementos que están en `c2` | Diferencia: $c - c2$ |
| `retainAll(c2)` | Se queda solo con lo que también está en `c2` | Intersección: $c \cap c2$ |
| `size()` / `isEmpty()` | Número de elementos / ¿está vacía? | — |

# 4. Listas: `ArrayList` vs. `LinkedList`

Ambas implementan `List<T>`, pero por dentro funcionan de forma muy distinta:

| | **`ArrayList`** | **`LinkedList`** |
|---|---|---|
| Por dentro | Un array que crece dinámicamente | Nodos enlazados entre sí (referencias) |
| Acceso por índice `get(i)` | Rápido (directo) | Lento (recorre nodo a nodo) |
| Insertar/quitar al principio o en medio | Lento (desplaza el resto) | Rápido (solo cambia referencias) |

| | `ArrayList` | `LinkedList` |
|---|---|---|
| Cuándo usarla | Accedes mucho por posición, casi no insertas en medio | Insertas/quitas mucho en los extremos o en medio |

```
List<String> nombres = new ArrayList<>();
nombres.add("Ada");
nombres.add("Alan");
nombres.get(0);          // "Ada"
nombres.remove("Alan");
```

# 5. Conjuntos: `HashSet` vs. `TreeSet`

- **`HashSet`**: no garantiza ningún orden, pero `add`/`contains`/`remove` son muy rápidos (usa el `hashCode()` de los elementos — de ahí la importancia de implementarlo bien, como viste en la unidad 4).
- **`TreeSet`**: mantiene los elementos **siempre ordenados** (necesita que los elementos implementen `Comparable`, o que le pases un `Comparator`), a cambio de operaciones algo más lentas.

```
Set<String> nombres = new HashSet<>();
nombres.add("Ada");
nombres.add("Ada");        // se ignora: ya estaba
System.out.println(nombres.size());   // 1
```

# 6. Colas y pilas: `Queue` y `Deque`

- **`Queue<T>`** (cola, FIFO — el primero en entrar es el primero en salir): `offer(e)` añade, `poll()` saca y devuelve el primero.
- **`Deque<T>`** (cola de dos extremos) también sirve como **pila** (LIFO — el último en entrar es el primero en salir), con `push(e)`/`pop()`.

```java
Deque<Integer> pila = new ArrayDeque<>();
pila.push(1);
pila.push(2);
System.out.println(pila.pop());   // 2 — el último en entrar, el primero en salir
```

# 7. Mapas: `HashMap` y `TreeMap`

Un `Map<K,V>` asocia claves únicas a valores — como un diccionario.

```java
Map<String, Integer> edades = new HashMap<>();
edades.put("Ada", 28);
edades.put("Alan", 41);
edades.get("Ada");                  // 28
edades.getOrDefault("Eva", 0);      // 0, porque "Eva" no está
edades.containsKey("Alan");         // true

for (Map.Entry<String, Integer> entrada : edades.entrySet()) {
    System.out.println(entrada.getKey() + ": " + entrada.getValue());
}
```

Igual que con los conjuntos, `HashMap` no garantiza orden pero es muy rápido, y `TreeMap` mantiene las claves ordenadas a cambio de algo más de coste.

# 8. Iteradores

Para recorrer cualquier colección sin importar su implementación interna, usas un `Iterator<T>`:

```java
Iterator<Integer> it = lista.iterator();
while (it.hasNext()) {
    int valor = it.next();
    if (valor % 2 != 0) {
        it.remove();   // elimina el elemento actual de forma segura
    }
}
```

⭐ **Be the Code**: `it.remove()` es la **única** forma segura de eliminar elementos de una colección mientras la recorres. Si intentas `lista.remove(...)` directamente dentro de un `for-each`, obtienes un `ConcurrentModificationException` — el propio `for-each` usa un iterador por debajo, y modificar la colección "por otro lado" mientras se recorre lo deja en un estado inconsistente.

El bucle `for (T x : coleccion)` que ya conoces es, por dentro, exactamente esto: azúcar sintáctico sobre un `Iterator`.

# 9. Ordenar objetos: `Comparable` y `Comparator`

Para que Java sepa ordenar tus propios objetos (en un `TreeSet`, con `Collections.sort(...)`, etc.), necesita saber comparar dos de ellos.

**`Comparable<T>`**: la clase define su **orden natural**, implementando `compareTo`:

```
class JugadorFutbol implements Comparable<JugadorFutbol> {
    private String nombre;
    private int goles;

    @Override
    public int compareTo(JugadorFutbol otro) {
        return Integer.compare(this.goles, otro.goles);   // orden natural: por goles
    }
}

List<JugadorFutbol> jugadores = new ArrayList<>();
Collections.sort(jugadores);   // usa compareTo()
```

**`Comparator<T>`**: un objeto **externo** que define un orden alternativo, sin tocar la clase — útil cuando necesitas ordenar por distintos criterios en distintos momentos:

```
Comparator<JugadorFutbol> porNombre = (j1, j2) ->
j1.getNombre().compareTo(j2.getNombre());
jugadores.sort(porNombre);   // ordena por nombre, sin depender del compareTo() de la
clase
```

`compareTo`/`compare` devuelven: **negativo** si el primero es “menor”, **cero** si son “iguales” a efectos de orden, **positivo** si el primero es “mayor”.

# 10. Crear tus propias colecciones

Las colecciones de Java cubren la inmensa mayoría de casos, pero entender cómo funcionan por dentro te ayuda a usarlas mejor. Una **lista enlazada** propia, en su forma más simple, es una cadena de nodos, cada uno con un dato y una referencia al siguiente:

```
class Nodo<T> {
    T dato;
    Nodo<T> siguiente;
    Nodo(T dato) { this.dato = dato; }
}
```

Encadenando nodos (`nodo1.siguiente = nodo2`, y así sucesivamente) construyes tu propia lista, sin usar arrays por debajo — exactamente la idea en la que se basa `LinkedList`. Otras estructuras de datos que no vienen prefabricadas en el paquete estándar (como los **árboles** — cada nodo con varios “hijos” en vez de un único “siguiente” — o los **grafos**) se construyen con la misma idea: nodos con referencias entre sí.

# RA y CE que cubre esta unidad

| RA | Criterios de evaluación cubiertos |
|---|---|
| RA6. Escribe programas que manipulen información con tipos avanzados de datos… | a) programas con arrays · b) librerías de tipos avanzados · c) listas · d) iteradores · e) características y ventajas de las colecciones · f) clases y métodos genéricos |

🟢 Boletín inicial ⭐ Boletín intermedio 🔥 Extras